

OOP 練習 4：靜態 vs 實例

學習目標

- 會宣告靜態成員：**CLASS-DATA** (類別層級的屬性)、**CLASS-METHODS** (不用 **NEW** 就能呼叫的方法)
- 會用 **=>** 呼叫靜態成員 (對照實例的 **->**)
- 理解 **CLASS-DATA** 是「整個類別共用一份」，所有物件看到同一個值
- 能判斷使用時機：純計算、無狀態的工具 → 靜態；每個物件要記自己的資料 → 實例
- 知道規則：實例方法可以碰靜態成員，靜態方法碰不到實例成員

事前準備

建立程式 **ZR_0004_<你的姓名縮寫>**，套件 **\$TMP**。(本題是 local class 的最後一題，op05 起改用 ADT 寫全域類別)

題目需求

1. 工具類 **lcl_flight_util**：全靜態、不記狀態
 - **CLASS-METHODS** **occupancy_rate**：IMPORTING **iv_seatsmax** TYPE **s_seatsmax**、**iv_seatsocc** TYPE **s_seatsocc**，RETURNING 載客率 (百分比，1 位小數；**iv_seatsmax** 為 0 時回 0，避免除零)
 - 主程式不 **NEW**，直接 **lcl_flight_util=>occupancy_rate(...)** 計算 340 座賣出 290 座的載客率
2. 有狀態的類別 **lcl_booking**：
 - **CLASS-DATA** **gv_created** TYPE **i** READ-ONLY：記錄「總共建立過幾個訂位物件」
 - **constructor**：IMPORTING **iv_carrid**，並把 **gv_created +1**
 - 主程式 **NEW** 三個物件 (**AA**、**LH**、**SQ**)，輸出 **lcl_booking=>gv_created** 驗證是 3
 - 再用其中一個物件 **go_b1->gv_created** 讀同一個值——證明「共用一份」
3. 實驗：在 **occupancy_rate** 裡故意引用一個 **mv_** 實例屬性，看編譯錯誤訊息 (看完拿掉)

預期輸出 (範例)

```
=== 靜態方法：不用 NEW，用 => 直接呼叫 ===
AA 0017 載客率：85.3 %
=== CLASS-DATA：所有物件共用一份 ===
已建立的訂位物件數：3
( go_b1 看到的也是同一份：3 )
```

團隊實務備註

- 靜態屬性的前綴慣例沿用 **gv_/gt_** (類別層級只有一份，性質同全域)，對照實例的 **mv_/mt_**
- 工具類 (全靜態) 很方便，但別濫用：靜態方法無法被子類別重新定義、難以在單元測試中抽換 (op10 測試、op08 多型之後回頭就懂)——會長狀態、會有變化的邏輯還是用實例
- **CLASS-DATA** 的初始化時機就是 op03 教的 **class_constructor**——兩者是天生一對

思考題

1. op02 的 `calc` (營收計算) 完全無狀態，把它改成 `CLASS-METHODS` 有什麼好處？又會失去什麼？
(op02 思考題 2 的正式解答)
2. `gv_created` 現在只增不減——如果想在物件被回收時 -1，ABAP 有 destructor 嗎？(查查看，答案會讓你意外)
3. 為什麼靜態方法碰不到 `mv_` 實例屬性？從「靜態方法執行時，根本沒有物件」的角度想。

答案

見 `zr_oo04_static.prog.abap` (SAP 端程式 `ZR_0004_STATIC`)。